

Les transactions et les événements MYSQL

Les transactions dans MYSQL

Introduction :

Par défaut, MySQL valide automatiquement les modifications de manière permanente dans la base de données. Pour forcer MySQL à ne pas valider automatiquement les modifications, utiliser l'instruction suivante :

```
SET autocommit = 0;  
# Ou alors :  
SET autocommit = OFF;
```

Et pour activer explicitement le mode autocommit utiliser l'instruction suivante :

```
SET autocommit = 1;  
# Ou alors :  
SET autocommit = ON;
```

Une transaction dans MySQL est **un groupe séquentiel d'instructions** telles que **INSERT**, **UPDATE** et **DELETE** à exécuter comme une seule unité de travail qui peut être validée (**COMMIT**) ou annulée (**ROLLBACK**).

Si la transaction effectue plusieurs modifications dans la base de données, deux choses se produisent :

- Soit toutes les modifications sont réussies lorsque la transaction est validée.
- Ou, toutes les modifications sont annulées lorsque la transaction est annulée.

Définir une transaction :

Pour démarrer explicitement un bloc de transaction deux commandes sont possibles :

```
# Syntaxe 1 :  
BEGIN;  
statements  
COMMIT/ROLLBACK;  
  
# Syntaxe 2 :  
START TRANSACTION;  
statements  
COMMIT/ROLLBACK;
```

Les commandes suivantes sont utilisées pour contrôler les transactions :

- **COMMIT** : Pour enregistrer et valider les modifications.
- **ROLLBACK** : Pour annuler les modifications.

Verrouillage des données d'une transaction:

Au moment de la création d'une transaction, les données manipulées par cette transaction sont verrouillées jusqu'à leurs validation ou annulation.

Exemple 1 :

```
USE w3schools;  
SELECT * FROM products;
```

	ProductID	ProductName	SupplierID	CategoryID	Unit	Price
▶	1	Chais	1	1	10 boxes x 20 bags	18
	2	Chang	1	1	24 - 12 oz bottles	19
	3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
	4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22
	5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35
	6	Grandma's Boysenberry Spread	3	2	12 - 8 oz jars	25

```
# SESSION 1  
BEGIN;  
UPDATE products SET Price=100 WHERE ProductID= 1;  
# Transaction non encore validée  
SELECT * FROM products;
```

	ProductID	ProductName	SupplierID	CategoryID	Unit	Price
▶	1	Chais	1	1	10 boxes x 20 bags	100
	2	Chang	1	1	24 - 12 oz bottles	19
	3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
	4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22
	5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35
	6	Grandma's Boysenberry Spread	3	2	12 - 8 oz jars	25
	7	Uncle Bob's Organic Dried Pears	3	7	12 - 1 lb pkgs.	30

```
# SESSION 2  
SELECT * FROM products;  
UPDATE products SET Price=200 WHERE ProductID= 1;
```

	ProductID	ProductName	SupplierID	CategoryID	Unit	Price
▶	1	Chais	1	1	10 boxes x 20 bags	18
	2	Chang	1	1	24 - 12 oz bottles	19
	3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
	4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22
	5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35
	6	Grandma's Boysenberry Spread	3	2	12 - 8 oz jars	25

Output				
Action Output				
	#	Time	Action	Message
✓	1	17:56:05	SELECT * FROM products LIMIT 0, 1000	81 row(s) returned
4	2	17:56:05	UPDATE products SET Price=200 WHERE ProductID= 1	Running...

```
# SESSION 1
COMMIT;
# SESSION 2
SELECT * FROM products;
```

	ProductID	ProductName	SupplierID	CategoryID	Unit	Price
►	1	Chais	1	1	10 boxes x 20 bags	100
	2	Chang	1	1	24 - 12 oz bottles	19
	3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
	4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22
	5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35
	6	Grandma's Boysenberry Spread	3	2	12 - 8 oz jars	25
	7	Uncle Bob's Organic Dried Pears	3	7	12 - 1 lb pkgs.	30

Valider une transaction :

L'instruction **COMMIT** est utilisée pour valider une transaction.

Exemple 2 :

```
# SESSION 1
BEGIN;
UPDATE products SET Price=18 WHERE ProductID= 1;
COMMIT;
SELECT * FROM products;
```

	ProductID	ProductName	SupplierID	CategoryID	Unit	Price
►	1	Chais	1	1	10 boxes x 20 bags	18
	2	Chang	1	1	24 - 12 oz bottles	19
	3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
	4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22
	5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35
	6	Grandma's Boysenberry Spread	3	2	12 - 8 oz jars	25

```
# SESSION 2
SELECT * FROM products;
```

	ProductID	ProductName	SupplierID	CategoryID	Unit	Price
►	1	Chais	1	1	10 boxes x 20 bags	18
	2	Chang	1	1	24 - 12 oz bottles	19
	3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
	4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22
	5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35
	6	Grandma's Boysenberry Spread	3	2	12 - 8 oz jars	25

Annuler une transaction :

L'instruction **ROLLBACK** est utilisée pour annuler une transaction.

Exemple 3 :

```
# SESSION 1
BEGIN;
UPDATE products SET Price=100 WHERE ProductID= 1;
ROLLBACK;
SELECT * FROM products;
```

	ProductID	ProductName	SupplierID	CategoryID	Unit	Price
▶	1	Chais	1	1	10 boxes x 20 bags	18
	2	Chang	1	1	24 - 12 oz bottles	19
	3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
	4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22
	5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35
	6	Grandma's Boysenberry Spread	3	2	12 - 8 oz jars	25

```
# SESSION 2
SELECT * FROM products;
```

	ProductID	ProductName	SupplierID	CategoryID	Unit	Price
▶	1	Chais	1	1	10 boxes x 20 bags	18
	2	Chang	1	1	24 - 12 oz bottles	19
	3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
	4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22
	5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35
	6	Grandma's Boysenberry Spread	3	2	12 - 8 oz jars	25

Cas pratique :

Comme exemple plus concret, imaginons que nous tentons de transférer 2000 DH d'un compte à un autre.

On commence par la création de la base de données :

```
CREATE DATABASE DEMO;
USE DEMO;
CREATE TABLE accounts
(id INT PRIMARY KEY,
name VARCHAR(50),
balance DOUBLE
);
INSERT INTO accounts VALUES(1,'account1',25000);
INSERT INTO accounts VALUES(2,'account2',1000);
SELECT * FROM accounts;
```

	id	name	balance
▶	1	account1	25000
	2	account2	1000
*	NULL	NULL	NULL

Ici, les 2000 DH ne seront pas retirés du compte avec id = 1 sans mettre également 2000 DH sur le compte avec id = 2 :

```
BEGIN;  
  UPDATE accounts SET balance = balance - 2000 WHERE id=1;  
  UPDATE accounts SET balance = balance + 2000 WHERE id=2;  
  COMMIT;  
  
SELECT * FROM accounts;
```

	id	name	balance
▶	1	account1	23000
	2	account2	3000
*	NULL	NULL	NULL

On suppose qu'après avoir émis les instructions **UPDATE** nous avons accidentellement transféré le mauvais montant ou utilisé les mauvais comptes, nous pourrions annuler les modifications au lieu de les valider :

```
INSERT INTO accounts VALUES(3,'account3',0);  
SELECT * FROM accounts;  
  
START TRANSACTION;  
  UPDATE accounts SET balance = balance - 1000 WHERE id=1;  
  UPDATE accounts SET balance = balance + 1000 WHERE id=3;  
  ROLLBACK;  
  
SELECT * FROM accounts;
```

	id	name	balance
▶	1	account1	23000
	2	account2	3000
	3	account3	0
*	NULL	NULL	NULL

Les événements planifiés MYSQL

Introduction :

Les événements MySQL sont des tâches qui s'exécutent selon un calendrier spécifié. Par conséquent, les événements MySQL sont parfois appelés événements planifiés.

Les événements MySQL sont des objets nommés qui contiennent une ou plusieurs instructions SQL. Ils sont stockés dans la base de données et exécutés à un ou plusieurs intervalles.

Par exemple, vous pouvez créer un événement qui optimise toutes les tables de la base de données et qui s'exécute à 1 h 00 tous les dimanches.

Configuration du planificateur d'événements MySQL :

MySQL utilise un thread spécial appelé thread du planificateur d'événements pour exécuter tous les événements planifiés. Vous pouvez afficher l'état du thread du planificateur d'événements en exécutant la commande :

```
SHOW PROCESSLIST;
```

Créer de nouveaux événements MySQL :

Voici la syntaxe de base pour créer un événement dans MYSQL :

```
CREATE EVENT [IF NOT EXIST] event_name  
ON SCHEDULE your_schedule  
DO  
event_body
```

Dans cette syntaxe :

- Tout d'abord, indiquer le nom de l'événement **event_name** dont vous souhaitez créer. Les noms d'événements doivent être uniques dans la même base de données.
- Deuxièmement, spécifier un calendrier après le mot-clé **ON SCHEDULE**.
- Le **your_schedule** est l'heure à laquelle les instructions doivent être exécutées.
- Le **event_body** est l'ensemble d'instructions à exécuter.

your_schedule :

Si l'événement est un événement ponctuel, utilisez la syntaxe :

```
AT time_stamp [+ INTERVAL your_interval]
```

Si l'événement est un événement récurrent, utilisez la clause **EVERY** :

```
EVERY your_interval [STARTS time_stamp [+ INTERVAL your_interval]  
[ENDS time_stamp [+ INTERVAL your_interval]
```

your_interval

```
quantity {YEAR | MONTH | DAY | HOUR | MINUTE | WEEK | SECOND ... }
```

Exemple d'événement ponctuel :

L'exemple suivant crée un événement ponctuel qui insère une nouvelle ligne dans une table :

```
CREATE TABLE messages (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  message VARCHAR(255) NOT NULL,  
  created_at DATETIME NOT NULL  
);  
  
CREATE EVENT event_01  
ON SCHEDULE AT CURRENT_TIMESTAMP  
DO  
  INSERT INTO messages(message,created_at)  
  VALUES ('Test MySQL Event 1',NOW());  
  
SELECT * FROM messages;
```

Pour afficher tous les événements de la base de données, utilisez l'instruction suivante :

```
SHOW EVENTS;
```

	Db	Name	Definer	Time zone	Type	Execute at	Interval value	Interval field	Starts	Ends	Status	Originator	character_set_client	collation_connection	Database Collation

La sortie n'affiche aucune ligne car l'événement est automatiquement supprimé car par l'événement expire après son exécution.

Pour conserver l'événement après son expiration, utiliser la clause **ON COMPLETION PRESERVE**.

```
CREATE EVENT event_02  
ON SCHEDULE AT CURRENT_TIMESTAMP + INTERVAL 1 MINUTE  
ON COMPLETION PRESERVE  
DO  
  INSERT INTO messages(message,created_at)  
  VALUES ('Test MySQL Event 2',NOW());  
  
CREATE EVENT event_03  
ON SCHEDULE AT '2023-02-03 23:46:00'  
ON COMPLETION PRESERVE  
DO  
  INSERT INTO messages(message,created_at)  
  VALUES ('Test MySQL Event 3',NOW());
```

Exemple d'événement récurrent :

L'exemple suivant crée un événement récurrent qui s'exécute toutes les minutes et expire 1 heure après sa création :

```
CREATE EVENT event_04
ON SCHEDULE EVERY 1 minute
STARTS CURRENT_TIMESTAMP
ENDS CURRENT_TIMESTAMP + INTERVAL 1 HOUR
DO
    INSERT INTO messages(message,created_at)
    VALUES ('Test MySQL Event 4',NOW());
```

Supprimer un événement:

Voici la syntaxe pour supprimer un événement MYSQL :

```
DROP EVENT [IF EXIST] event_name;
```

Exemple :

```
DROP EVENT [IF EXIST] event_03;
```